

Debater Memory — Implementation Notes

AI Debate Arena · what "agentic memory" means here and exactly what changed in the code

The idea

Each debate resolution now has its own persistent memory, split per side. When a debate finishes, the judge's own scorecard (which already grades five criteria — Logic, Evidence, Rebuttal, Clarity, Persuasion — with a short reason per criterion) is mined for lessons: the weakest 1–2 criteria become "work on" notes, the strongest becomes a "keep doing" note. Those notes are saved to the browser's `localStorage`, keyed by the exact resolution text and the side (`alpha` / `beta`).

The next time you start a debate with the **same resolution text**, each side's system prompt is prefixed with its own coaching notes from prior rounds on that topic before the first turn is generated. It doesn't touch scoring or rig the outcome — the judge still grades the new transcript from scratch, so a previously-losing side can improve and still lose again if the opponent argues better that round.

Scope, by design: memory is tied to the *exact topic text*, not a general skill that carries across different topics. Both sides accumulate memory every debate (not just the loser) — so winners reinforce what worked, losers get pointed at what didn't. There's no new UI for this yet; it works silently under the hood.

What was added

NEW FILE — `src/lib/debate/memory.ts`

A small, dependency-free module. No extra LLM calls — it derives lessons purely from data the judge already computed, so it works identically in Live and Simulation mode.

Export	Purpose
<code>normalizeTopic(topic)</code>	Lowercases/trims/collapses whitespace so retyping the same resolution reliably matches the same memory key.
<code>getMemory(topic, side)</code>	Reads that side's stored memory for a topic from <code>localStorage</code> , or <code>null</code> if none exists yet.
<code>recordDebateMemory(topic, scorecard)</code>	Called once per finished debate. Derives lessons for <i>both</i> sides from the scorecard and merges them into the persisted record (capped at the last 6 lessons per side).
<code>lessonsPromptBlock(memory)</code>	Renders a side's stored memory into a short coaching paragraph — <code>Work on: ... / Keep doing: ...</code> — ready to drop into a system prompt. Returns <code>""</code> if there's nothing yet.

Storage shape (`localStorage` key `debate-arena-memory-v1`):

```
{
  "hp laptops are better than dell laptops": {
    "alpha": { timesDebated: 2, lastResult: "tie", lessons: [...], updatedAt: ... },
    "beta": { timesDebated: 2, lastResult: "tie", lessons: [...], updatedAt: ... }
  }
}
```

What a stored lesson looks like

```
{ criterion: "Evidence", kind: "improve",
  text: "Evidence: 0 numeric/quantified references – more concrete data would strengthen the case." }
{ criterion: "Clarity", kind: "keep",
  text: "Clarity: Average sentence length 17 words – crisp and easy to follow on stage."
}
```

What changed

EDITED – `src/lib/debate/useDebate.ts`

- **Load memory at debate start.** In `start()`, both sides' memory for the typed topic is loaded into a ref (`memoryRef`), and a per-run "already recorded" flag is reset.
- **Inject into the system prompt.** `systemFor()` gained a `lessons` parameter; `runTurn()` now passes `lessonsPromptBlock(memoryRef.current[side])` into it, so the coaching block appears in the system message from each side's very first turn.
- **Record memory when the debate actually finishes.** Inside `judgeDebate()`, a guarded `maybeRecordMemory()` calls `recordDebateMemory()` — but only when: the scorecard is final (not an interim, mid-debate score), all rounds have actually been played, and it hasn't already recorded once for this run. This guard matters because the existing "Re-score" button can request a final-style scorecard at any point, including mid-debate or repeatedly after the fact — without it, memory could get written early or duplicated.
- **Reset cleanly.** `reset()` also clears the in-memory refs, so a fresh topic never carries over stale lessons from the previous one.

End-to-end flow

1. You type a resolution and hit **Start Debate**.
2. `useDebate` loads any existing memory for that exact topic, per side.
3. Each debater's system prompt is built with its stored lessons (if any) appended.
4. The debate runs; the judge scores it.
5. Once every round has been played and the judge returns a final verdict, that scorecard is mined into new lessons and merged into `localStorage` for both sides.

6. Reset and restart with the *same* resolution text → step 2 now finds the memory from step 5.

How it was verified

Ran the actual dev server and drove it with a headless browser (Playwright against system Edge) rather than just reading the code:

- Ran a full debate on *"HP laptops are better than Dell laptops"* in Simulation mode to completion and confirmed `debate-arena-memory-v1` appeared in `localStorage` with derived lessons for both sides.
- Clicked **Reset Arena**, retyped the identical resolution, and started a second debate.
- Opened the app's own Developer Console → Live HTTP monitor (which logs real outgoing request bodies) and confirmed both sides' opening requests contained `"You have argued this exact resolution before..."` plus their `Work on:` / `Keep doing:` notes from round 1.
- Let the second debate finish and confirmed `timesDebated` incremented to 2 with lessons merged in, not overwritten.

Typecheck and lint were run on the changed/new files; the only pre-existing errors found (a missing `voice` field in `judge.ts`, and repo-wide CRLF line-ending lint warnings) predate this change and were confirmed unrelated by diffing against the base branch.